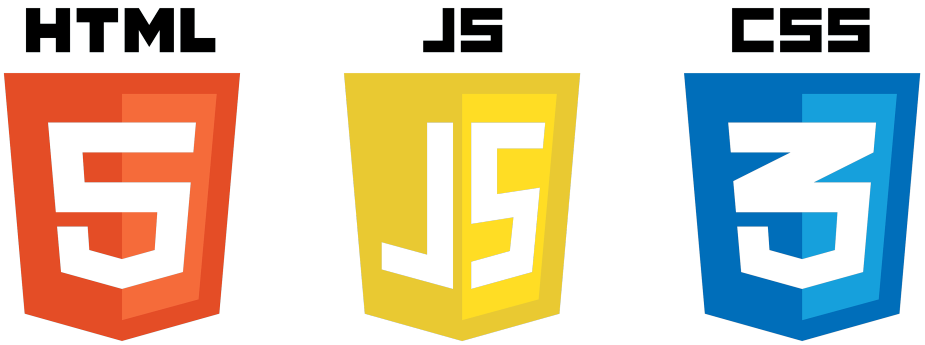
Data attributes, Event Bubbling en Delegation



Events

Events in JavaScript (herhaling)

Een event gebeurt wanneer een gebruiker iets doet
of wanneer de browser iets uitvoert, bijvoorbeeld:

- klikken op een knop
- typen in een inputveld
- over een element bewegen met de muis (hover)
- een toets indrukken
- webpagina volledig laden

In JavaScript reageren we op een event met een event listener.

```
button.addEventListener("click", () => {
    console.log("Klik!");
});
```

Het event object e

Bij een event listener kan je een parameter toevoegen.

```
button.addEventListener("click", (e) => {
    console.log(e);
});
```

Object e bevat informatie over het event, zoals:

- event type: click, dblclick, keydown, mouseenter, ... (`e.type`)
- element waar het event op gebeurt (`e.target`)
- waar er precies geklikt werd (`e.pageX`, `e.pageY`)

Data attributes

Wat zijn data attributes?

- HTML kan extra informatie opslaan in elementen
- Dit gebeurt met attributen die beginnen met `data-`
- JavaScript kan deze informatie uitlezen

Data attributes: voorbeeld

- HTML: data opslaan met `data-*`

```html
<button data-product="pizza" data-prijs="12">
    Bestel pizza (12 Euro)
</button>
```

- JavaScript: data uitlezen met `dataset`

```javascript
button.addEventListener("click", (e) => {
    const product = e.target.dataset.product;
    const prijs = e.target.dataset.prijs;
    console.log(product);  // pizza
    console.log(prijs);    // 12
});
```

Waarom data attributes gebruiken?

- Element-specifieke informatie moet zoveel mogelijk in HTML
- JavaScript enkel logica laten uitvoeren
- Meer herbruikbare code

Opdracht 1

Maak gebruik van data attributes om informatie van JavaScript naar HTML te verplaatsen.

- Kleuren `lightblue` en `lightgreen`
- Font size steps `-2` en `4`
  - Tip – cast een string naar getal in JS met `Number(...)`

Event bubbling

Event bubbling

Wanneer een event gebeurt op een element, wordt dat event niet alleen op dat element uitgevoerd.

Het event bubbelt omhoog door de DOM.

Bijvoorbeeld:

- een klik op een `button`
- gaat daarna naar de `div` eromheen
- daarna naar `body`
- uiteindelijk naar `document`

Event bubbling: voorbeeld

- HTML:

```
<div id="panel">
    <button>Klik op mij</button>
</div>
```

- Als je op de knop klikt:
  - eerst wordt het event uitgevoerd op de `button`
  - daarna op `div #panel`
  - daarna op de `body`
  - daarna op `document`

Het event beweegt van het kind-element naar de parent-elementen.

Event delegation

Probleem

Stel dat we veel knoppen hebben: bijvoorbeeld 20.

Voor elke knop moeten we een event listener toevoegen.

Nadelen:

- Veel event listeners
- Niet efficiënt
- Nieuwe knoppen krijgen geen listener

Oplossing: Event delegation

In plaats van listeners op elke button te plaatsen,
doen we event delegation:

- één listener op een parent plaatsen
- telkens controleren waarop er geklikt werd (met `e.target`)

Dit werkt dankzij event bubbling.

Event delegation: voorbeeld

- HTML

```html
<div id="panel">
    <button class="btn">Knop 1</button>
    <button class="btn">Knop 2</button>
</div>
```

- JavaScript

```javascript
const panel = document.querySelector("#panel");

panel.addEventListener("click", (e) => {
  if (!e.target.matches(".btn")) return;
  console.log("Knop geklikt:", e.target.textContent);
});
```

Opdracht 2

Maak gebruik van event delegation om één event listener te bekomen voor alle buttons.

- Tip – voorbeeld van een button:

```
<button class="btn"
        data-action="kleur"
        data-value="lightblue">
    Blauwe achtergrond
</button>
```

Opdracht 3 (extra)

Voeg een knop toe om een gele achtergrond te bekomen.

Vragen?

maxime.debosschere@ugent.be